

代码优化

mem2reg

这个是放在中端优化的最前边，在做 mem2reg 之前需要先分析数据，生成流图和支配树，具体的生成方法参考的是教程网站上给的算法。

之后就是 mem2reg 的步骤，首先需要构建 def-use，这个的 def use 是根据每条指令生成的，不仅包括 inst 还有 basicBlock。

之后是根据算法插入 phi 指令，简单来说就是根据支配边界将，后边的块插入一个 phi 指令，之后再通过 rename，将他的可能来源替换为前面块的 reachdef，这个 reachdef 的意思就是当前最近的定义，可以使用一个栈来处理。

死代码删除

这个我感觉我做的没问题，但实际用处不大，因为 mem2reg 已经做过一遍死代码删除了，而我又没有做其他的优化，就导致没效果。

死代码删除的原理是，根据每条有用的指令计算他的指令闭包，闭包就是他被谁用了，和用谁了，算完之后得到一个全部的集合，把除了集合之外的指令全删了就是死代码删除。

LVN

没做 gvn 的原因是，做了 gvn 还得做 gcm，做不完了。

lvn 很好写，就是根据每种指令设计一个 hash，然后在后边如果看到相同的 hash 就给替换了，如果是常量，可以直接利用 uselist 把后边指令中的 use 全给替换了。

在 lvn 的过程中可以顺便做一些简单的常量折叠，比如说两个操作数全都是常量的情况下，就可以直接计算出结果，如果是比较的逻辑，例如 $a * 0$ ，就可以直接用 0 来替换。

消除 PHI 指令

我放在了中端优化的最后面，实际上也就是后端的消除 phi。具体的操作来自于 SSA Book，这里记录一下实现的注意事项。

首先要对每个函数做插入 parallel copy，是并行赋值，意思就是在于将，phi 指令提前到当前基本块的前置基本块中，因为 parallel copy 是保证了并行赋值，所以暂时不需要移动。

这里要注意，因为根据 phi 指令的定义，phi 指令的 operand list 中的数量实际上是和他的前置基本块数量是一致，并且顺序也是一致的，因此这里可以直接将 phi 的 operand 放到 parallel copy 中。

```
// because phi's preBb is exactly the same as curBb's preBb
// it is under guarantee by the definition of phi instruction
// so parallel copy inst options' quantity is equal to phi instruction options'
quantity;
for (auto inst : curBb->instructionList) {
```

```

        if (inst->instType != InstType::PHI)
            break;

        for (int i = 0; i < pcList.size(); ++i) {
            pcList[i]->copyPair(inst, inst->operandList[i]);
        }

    }

    // kill all the phi Inst;
    curBb->instructionList.remove_if([](Instruction *i) { return i->instType
== InstType::PHI; });

```

同时在删除 phi 指令和插入 parallel copy 指令的时候也要记得维护 CFG 以保证之后不会出错。

```

/*
 * what we need to do here: (not necessarily in that order)
 *      ; update cfg info (done in insert bb)
 *      ; change branch instruction
 *      ; insert parallel copy
 */

// last instruction is branch, which is under guarantee by cfg;

```

完成了并行赋值之后，要进行 move 指令的插入，因为硬件上做不到同时赋值。

这里的修改也很简单，如果是既 def 又 use，就先放到一个 tmp 里边，不过会产生很多的 move 指令，这里多就多吧.....

图着色寄存器分配

这部分需要先通过 irParser 生成只有 虚拟寄存器的 Mips 指令流，然后根据后端的指令再次构建 def use，真的麻，在这还得再做一遍。

图着色的完全体是可以，计算预着色，和传送指令的，不过在竞速中，实际根本不会出现那么多的 spillNode，也就是说给你的寄存器绝大多数情况下都用不玩，于是我就没做预备着色和传送指令，将一些 可能会被用上的指令干脆不分配了

最后可以用的寄存器一共有 23 个。

图着色的原理就是根据上边提到的def use 构建冲突图，得到了一个全都是虚拟寄存器的图，这个时候挨个删，先删度小于寄存器数量的点，删不动了就随便拿出来一个当成 spillNode，毕竟不追求性能，之后将所有spillNode 的点用到的地方都变成内存的读取，最后根据用到的寄存器保存上下文 和调整栈的使用。

以下是我规划的栈的使用。

```

/*
 * ----- view of stack -----
 *
 * when entering a function
 *
 *      | # 7th arg
 *      | # 6th arg

```

```

*      | # 5th arg
*      ----- <-- $sp without fixing
*      |
*      | context saving where I'm planning
*      |
*      -----
*      |
*      |
*      | alloca
*      |
*      |
*      -----
*      $sp <--- here is stack pointer
*
* $sp is always at the bottom, in order to reduce calculation
*/

```

乘除法优化

我的乘法仅仅做了 当乘数为 2^n 的情况，除法 参考 **Division by Invariant Integers using Multiplication** 论文中的做法，复现算法。

指令优化

这个地方就是 后端 IRParser 的使用要简单一点，非常的琐碎，比如伪指令的选择等。